Assignment 2 – BFS and DFS Traversals

You've been hired to work at Amazon and need to develop pathfinding code to control Proteus warehouse robots that will deliver boxes from one arbitrary point in a warehouse to another arbitrary point using the shortest path.

This Assignment challenges you to solve two different 'maze' traversal algorithms. The traversal are *Breadth First Search* (BFS) and *Depth First Search* (DFS). Writing algorithms for traversals is a fun and challenging problem to solve.

Topics Covered

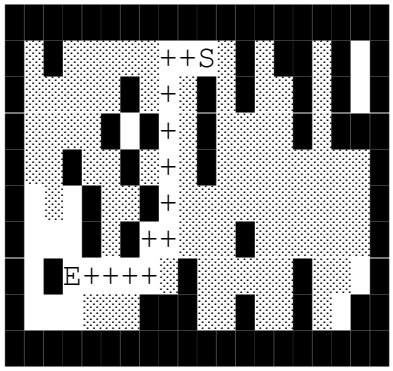
2D Arrays/Lists
Flood fill - https://en.wikipedia.org/wiki/Flood_fill
BFS - https://en.wikipedia.org/wiki/Breadth-first_search
DFS- https://en.wikipedia.org/wiki/Depth-first_search
Stack
Queue
Custom Objects
Parallel Arrays/Lists

Instructions

Describe the problem and explain your solution for each step. These notes can be compiled as documentation upon completion.

1. Generate a 2-dimensional array (grid) of points
2. Randomly fill the grid with obstacles
3. Randomly select a start and end point
4. Develop an algorithm that demonstrates a traversal using a Stack ADT
    a. This is considered a DFS
    b. You must employ a Stack in your solution
5. Develop an algorithm that will use the Stack flood fill data to find the shortest path from the start point to the end point
    a. This is called back-tracking
    b. This will construct the path
6. Develop an algorithm that demonstrates traversal using a Queue ADT
    a. This is considered a BFS
    b. You must employ a Queue in your solution
7. Develop an algorithm that will use the Queue flood fill data to find the shortest path from the start point to the end point
    a. This will construct the path

Sample Console Output



Groups

You are permitted to work with **one** other student and may submit a single solution. Please provide both names in your project documentation.

Tips

- Do not attempt this program in one day, work on it in stages
- 2D structures typically employ a nested loop patterns
- Work with another student on strategies, tips, etc. at a white-board level
- Set a goal and timeframe for each step in the instructions
- Each point in the grid must represent a ‘wall’ or ‘passage’, obstacle or empty, etc.
- Think of the 2D array as a bird-eye view of terrain, some points may be traversable, some may not
- Start with a very small and simple maze, consider parameterizing the width and height of the maze later
- You do not need to create a traditional looking maze, a simple grid with randomly placed obstacles will suffice
- Consider making a border of ‘walls’ to stay within the grid
- Consider using a debugger or creating a mechanism to step through a traversal
- Traversal pseudo code see Figure 8, 9, and 10 in the Appendix
- A simple way of traversing is accomplished by setting a ‘current position’, starting a loop, and updating the ‘current position’ inside of the loop
- Flood fill/traversing, ‘walk’ the grid and assign an incremented ‘distance’ value for each point that is encountered/traversable
- For back-tracking, start at the end point, and traverse the grid backwards by picking an adjacent point with the smallest flood fill value

Requirements

2D structure (10 points)
Random obstacles added (10 points)
Random start and end points added (10 points)
Traversal with Stack (10 points)

Path displayed from start to end (10 points)
Traversal with Queue (10 points)
Path displayed from start to end (10 points)
Documentation (10 points)
Source code cleanliness (10 points)

Constraints

You are only allowed to use the ADTs covered in class (up to Queues). If you wish to use an ADT other than what we covered in class, please ask the instructor before submitting it.

Do not use recursion.

Acceptable programming languages include Java, C++, C#, C, Rust, and Unity, and Unreal Engine. No Python solutions accepted.

Extra credit possibilities

1. Employ appropriate use of source control (+5)
2. Apply user-friendly GUI/menu to your solution (+5)
3. Employ multi-threading in your solution (+5)

Grading Rubric

Requirement	NA	F	C	B	A
2D structure		No apparent grid	2D structure apparent		2D structure with flexible points
Random obstacles added		All points in 2D structure traversable	Obstacles added to 2D structure		Border of obstacles added to 2D structure
Random start and end points added		No start and end point			Start and end point clearly marked
Traversal with Stack		Traversal passes through obstacles or goes out of bounds			Traversal stops at end point
Path displayed from start to end		No path displayed			Path displayed
Traversal with Queue		Traversal passes through obstacles or goes out of bounds			Traversal stops at end point
Path displayed from start to end		No path displayed			Traversal stops at end point
Documentation		Not documented well			Clear explanation of project goals, tasks, and solutions
Source code cleanliness		Inconsistent whitespace, variable and function names are difficult to understand, inefficient, unhelpful comments			Easy to read, variable and function names clear, efficient, helpful comments

Interview Grading Process

You are expected to have a deep understanding of the code you submitted as you were expect to write it. The questions asked will be tailored to your submission. If you understand your own program, it should be very easy to perform well.

I will ask 5 to 10 questions based on your solution. Some questions will be knowledge or design based. E.g *Why did you link four boolean expressions together in one if statement?* Most will be task based. E.g. *Make your solution always produce an unsolvable maze.* Tasks should be completed in under 2 minutes.

Each question/task relates to one of the Grading Rubric categories.

If a question/task is answered/complete, points will be awarded to the appropriate Grading Rubric category. If the question is unanswered or the task is incomplete, no points will be awarded for that category.

Academic Dishonesty

Reminder to read the syllabus about Academic Dishonesty.

Appendix

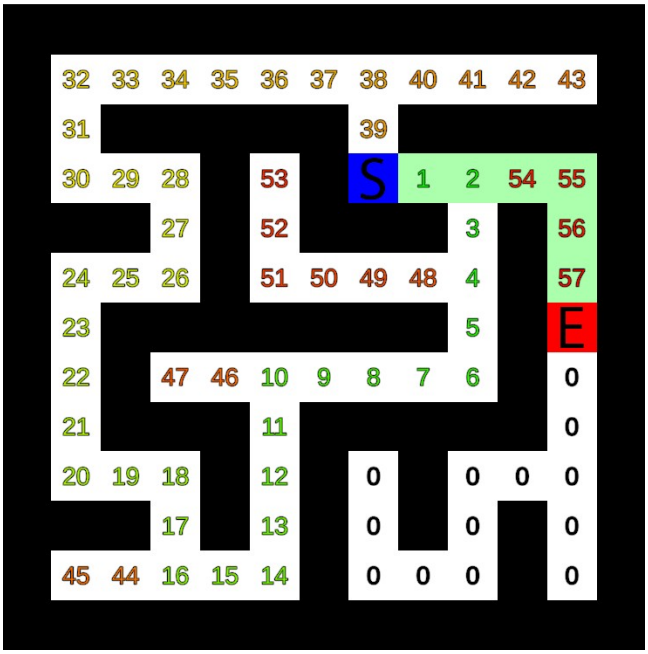


Figure 1. 2023 student submission DFS using a stack with traversal data visible. Built with Unity. Credit: emm45@pct.edu, (https://www.youtube.com/@wsalevan/featured)

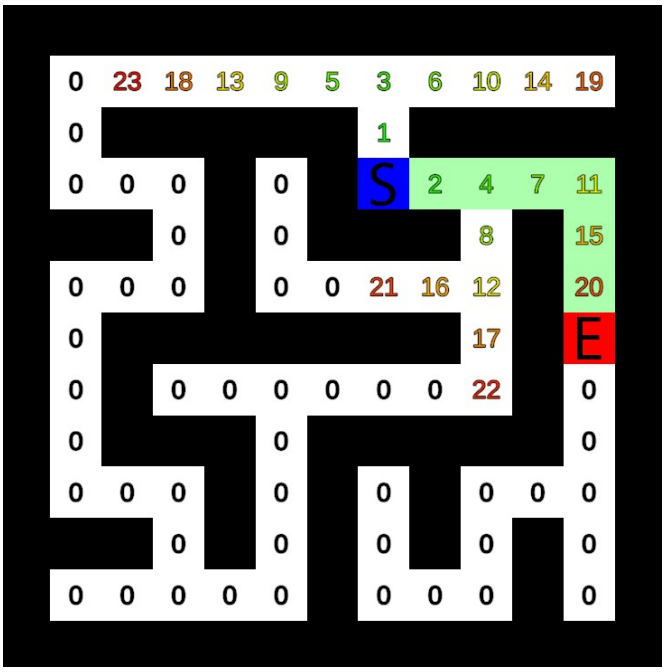


Figure 2. BFS using a queue with traversal data visible. Built with Unity. Credit: emm45@pct.edu, (https://www.youtube.com/@wsalevan/featured)

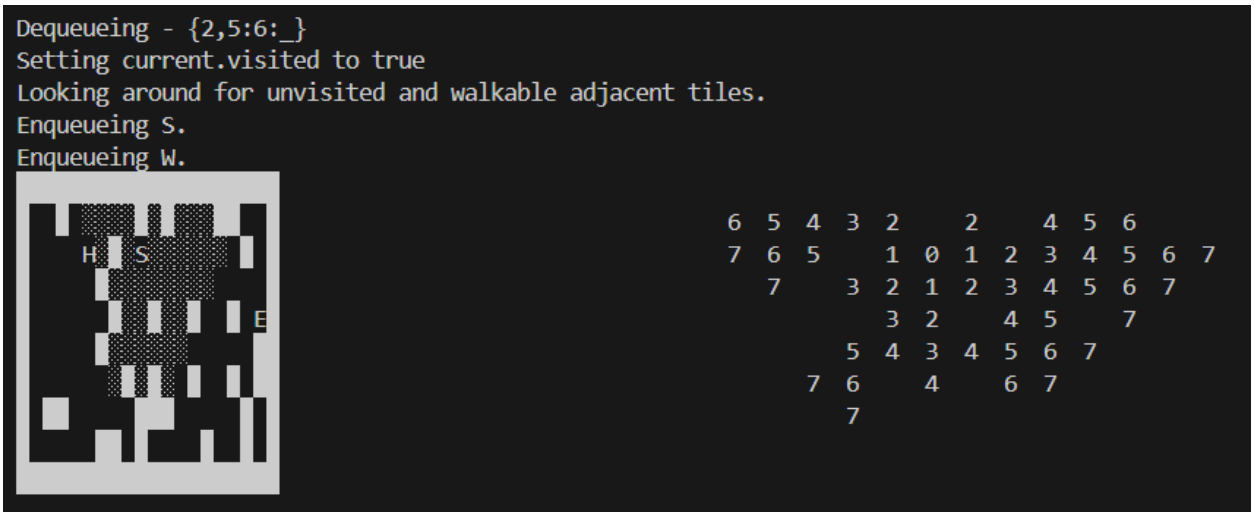


Figure 3. BFS Example without GUI. This is midway through flood fill. S is start point. E is Endpoint. H is current point being processed. Solid is walls. 'Gray' is visited. Data representation and debugging output is visible.

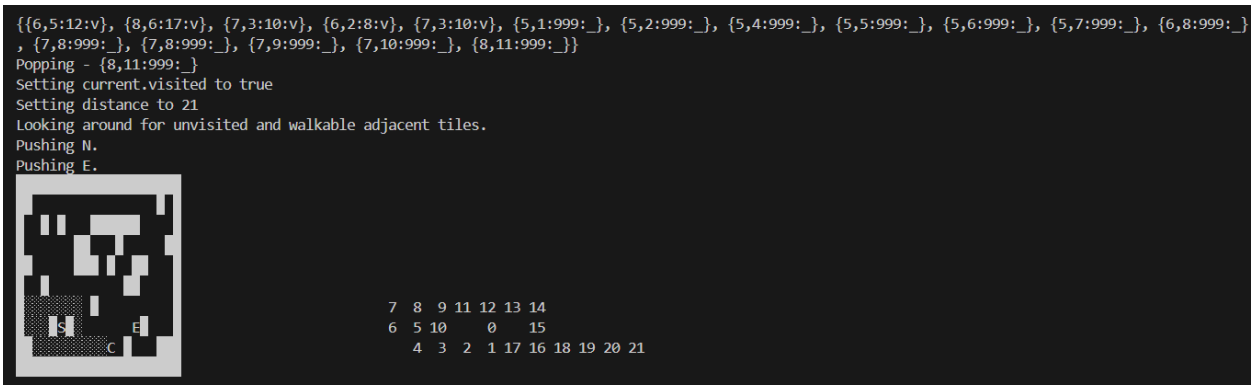


Figure 4. DFS Example without GUI. This is midway through flood fill. S is start point, E is end point, C is current point. Solid blocks are obstacles, 'Gray' is visited. Data representation of grid and debugging option enabled.

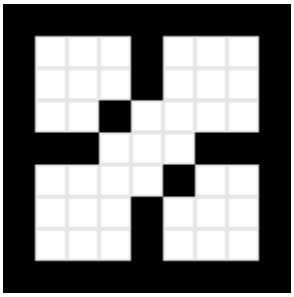


Figure 5. Flood fill with 4 directions, https://en.wikipedia.org/wiki/Flood_fill

```java
// Haversine function
// Copied from Geeks for Geeks 10/15/2024
// https://www.geeksforgeeks.org/haversine-formula-to-find-distance-between-two-points-on-a-sphere/
static double haversine(double lat1, double lon1, double lat2, double lon2){
    // distance between latitudes and longitudes
    double dLat = Math.toRadians(lat2 - lat1);
    double dLon = Math.toRadians(lon2 - lon1);

    // convert to radians
    lat1 = Math.toRadians(lat1);
    lat2 = Math.toRadians(lat2);

    // apply formulae
    double a = Math.pow(Math.sin(dLat / 2), 2) +
               Math.pow(Math.sin(dLon / 2), 2) *
               Math.cos(lat1) *
               Math.cos(lat2);
    double rad = 6371;
    double c = 2 * Math.asin(Math.sqrt(a));
    return rad * c;
}
```

Figure 6. Attribution example from Geeks for Geeks.

```python
# Haversine function
# taken from chatgpt.com
# 10/15/2024
# prompt: "print the haversine function in python"
def haversine(lon1, lat1, lon2, lat2):
    # Convert decimal degrees to radians
    lon1, lat1, lon2, lat2 = map(math.radians, [lon1, lat1, lon2, lat2])

    # Haversine formula
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = math.sin(dlat / 2)**2 + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2)**2
    c = 2 * math.asin(math.sqrt(a))

    # Radius of Earth in kilometers (use 3956 for miles)
    r = 6371

    # Calculate the result
    return c * r
```

Figure 7. Attribution example from ChatGPT.com.

```
DFS(G,v)   ( v is the vertex where the search starts )
   Stack S := {};   ( start with an empty stack )
   for each vertex u, set visited[u] := false;
   push S, v;
   while (S is not empty) do
      u := pop S;
      if (not visited[u]) then
         visited[u] := true;
         for each unvisited neighbour w of u
            push S, w;
      end if
   end while
END DFS()
```

Figure 8. Pseudo code of a DFS using a stack. Credit: Danny Heap, University of Toronto, https://www.cs.toronto.edu/~heap/270F02/node36.html

```
procedure BFS(G,s)

    for each vertex v ∈ V[G] do
        explored[v] ← false
        d[v] ← ∞
    end for
    explored[s] ← true
    d[s] ← 0
    Q:= a queue data structure, initialized with s
    while Q ≠ ϕ do
        u ← remove vertex from the front of Q
        for each v adjacent to u do
            if not explored[v] then
                explored[v] ← true
                d[v] ← d[u] + 1
                insert v to the end of Q
            end if
        end for
    end while

end procedure
```

Figure 9. Pseudo code of BFS using a Queue. Parallel arrays used. Credit: University of Maryland, https://www.cs.umd.edu/class/spring2021/cmsc351-0201/files/bfs.pdf

```
1  procedure BFS(G, root) is
2      let Q be a queue
3      label root as explored
4      Q.enqueue(root)
5      while Q is not empty do
6          v := Q.dequeue()
7          if v is the goal then
8              return v
9          for all edges from v to w in G.adjacentEdges(v) do
10             if w is not labeled as explored then
11                 label w as explored
12                 w.parent := v
13                 Q.enqueue(w)
```

Figure 10. Pseudo code of BFS using a Queue. Credit: Wikipedia, https://en.wikipedia.org/wiki/Breadth-first_search

			X	X		
		3	2	X		
	3	2	1	X	3	
3	2	1	0	1	2	3
	3	2	1	2	3	
		3	2	3		
			3			

Figure 11. Sample of BFS data. Dr. Bahram Golshan.

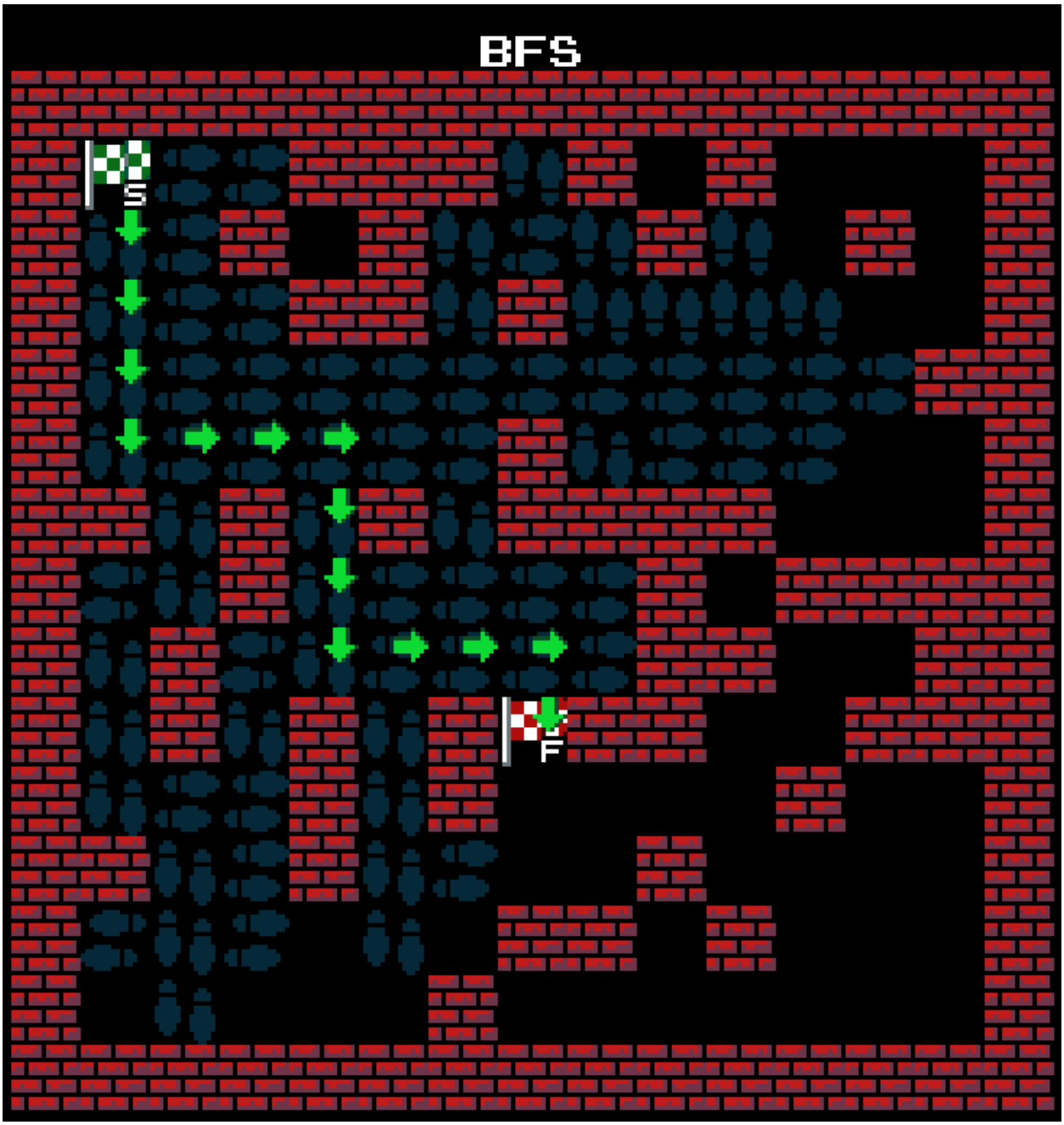


Figure 12. Sample Student submission. Credit ldm18@pct.edu, https://www.youtube.com/@wickingpedia1172/featured